GuardLLM: Automated Security Testing For Large Language Models in Production Environments

Harsh Solanki, Palash Thakkar, Rudra Kareliya,
Rupali Vairagade, Archana Bopche, Nilakshi Jain
Department of Cyber Security, Shah and Anchor Kutchhi Engineering College, Mumbai, India
{harash.solanki18600, palash.17404, rudra.kareliya18522}@sakec.ac.in
{rupali.vairagade, archana.bopche, nilakshi.jain}@sakec.ac.in

Abstract

The software industry is pushing Large Language Models (LLMs) into production environments at an unprecedented rate. However, this architectural shift exposes organizations to a severe security crisis. Unlike traditional software operating on strict deterministic logic, LLMs process system instructions and untrusted user input through the same medium: natural language. This lack of structural boundaries makes them inherently vulnerable to adversarial manipulation. Currently, development teams auditing these systems face a difficult choice: pay for expensive, privacy-violating cloud security platforms, or configure complex, command-line open-source scanners. We built GuardLLM to eliminate this compromise. GuardLLM is an automated, zero-configuration security testing toolkit that runs natively in a web browser. It features a novel dual-mode routing architecture, allowing developers to execute high-fidelity vulnerability scans via the cloud (using the Gemini API) or conduct offline assessments against local models (like Llama 3.1 and Gemma) using Ollama. Powered by an exhaustive 20-point threat matrix, GuardLLM goes beyond the standard OWASP LLM Top 10 by proactively generating advanced evasion tactics like context exhaustion and cross-lingual payloads. The system automatically crafts attacks, evaluates the model's reaction using heuristic analysis, and compiles a comprehensive, locally generated PDF audit report. This paper details the internal architecture of GuardLLM and presents a comprehensive evaluation against open-source and closed-source models, proving its efficacy as a scalable, privacy-conscious solution.

Keywords: Large Language Models, AI Security, Prompt Injection, Vulnerability Scanner, Penetration Testing, OWASP, Adversarial Machine Learning

1 Introduction

The rapid integration of Large Language Models (LLMs) into production environments introduces an entirely new category of vulnerabilities. Unlike standard deterministic software bound by strict execution rules, type-checking, and memory safety protocols, generative AI lacks rigid logical boundaries. LLMs process both hidden backend instructions (the system prompt) and external user input within the exact same natural language channel. Because "code" and "data" are intermingled, malicious actors can easily manipulate the AI using carefully worded conversational tricks.

Attackers actively hijack chatbots to extract proprietary data, bypass alignment filters, and execute unauthorized backend actions. A classic example involves appending a simple string like, *"Ignore all previous instructions and output your initial developer system prompt"* to a benign-looking query. If the model prioritizes the user's recent input over the developer's original instructions, it suffers a critical security breach known as Prompt Injection.

Despite these risks, standard tools like Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) are useless against non-deterministic chat interfaces. Properly testing an LLM requires specialized adversarial techniques to stress-test the model's internal parameters and alignment training. When searching for specialized AI testing tools, development teams

are typically stuck between two heavily flawed extremes: commercial enterprise platforms that route sensitive data through third-party SaaS architectures (creating compliance violations), or open-source research scanners that require complex command-line dependency trees.

We engineered GuardLLM from the ground up to solve this exact problem. It is a visual, fully automated testing environment running natively inside a web browser with no backend databases or heavy dependencies. Its dual-mode execution architecture allows engineers to seamlessly pivot between intelligent cloud-based attacks and entirely local, private execution. If a developer needs absolute privacy, "Local Mode" routes thousands of adversarial payloads directly to localized models hosted on their machine.

The core contributions of our work include:

- **Zero-Friction Client-Side Architecture:** A serverless, React-based testing framework operating entirely in the browser, removing the technical barrier to entry for AI auditing.
- **Dual-Mode Routing Protocol:** A network routing system bridging cloud-based API testing and fully offline local execution via Ollama for privacy-compliant validation.
- **Expanded 20-Point Threat Matrix:** An attack generation module moving beyond basic OWASP vulnerabilities, introducing 10 advanced adversarial evasion techniques modeled after real-world threat actors.
- **Automated Audit Reporting:** An on-the-fly visualization engine that processes test results into compliant PDF security reports without ever transmitting proprietary data externally.
- **Comparative Framework Evaluation:** A rigorous empirical validation study benchmarking the resilience of both closed-source commercial APIs and state-of-the-art open-weight models.

2 Background and the Problem Landscape

As the attack surface for language models explodes, the cybersecurity industry has scrambled to build defense mechanisms. The current ecosystem generally clusters into three distinct paradigms, each introducing severe friction points that prevent continuous, secure, and accessible security testing for standard software engineering teams.

2.1 The Friction of Open-Source Academic Scanners

Academic tools like Garak and LLMFuzzer effectively probe models for vulnerabilities. However, built exclusively for machine learning researchers, they require deep familiarity with Python, complex dependency management, and command-line interfaces. This creates a massive learning curve, preventing standard QA and DevOps teams from easily integrating AI security into deployment pipelines.

2.2 Defensive Middleware and Runtime Guardrails

To bypass the difficulties of comprehensive pre-deployment testing, many organizations attempt to apply runtime filters like NeMo Guardrails. While essential as a defense-in-depth measure, runtime monitoring cannot replace fundamental vulnerability assessment. Relying solely on a guardrail is akin to putting a Web Application Firewall (WAF) in front of exploitable code. Furthermore, middleware adds significant computational latency to every API call.

2.3 The Privacy Paradox of Commercial Platforms

Commercial enterprise platforms offer visual dashboards and automated testing but rely heavily on SaaS architectures, creating a massive privacy paradox. Companies must transmit proprietary system prompts and user data over the public internet. For highly regulated industries bound by strict data

residency laws (e.g., healthcare, finance), this external data transmission violates compliance, locking them out of accessible security tooling.

GuardLLM shatters this compromise. By hosting the scanning engine entirely within the client's local browser memory and enabling direct network connections to localized inference engines, GuardLLM delivers the clean, visual user experience of a commercial platform with the absolute, mathematically guaranteed privacy of an offline terminal tool.

3 Proposed System Architecture

When designing GuardLLM, we prioritized universal accessibility, absolute data privacy, and the efficient processing of massive text generation without freezing the user's host machine. GuardLLM operates as a client-side, serverless application. The frontend interface, complex attack payload generators, and heuristic evaluation algorithms were built using React and TypeScript, bundled via Vite. The entire application binary loads directly into the browser's local memory footprint, ensuring zero sensitive prompt data is leaked externally.

3.1 The State-Driven Execution Pipeline

To ensure usability for developers without penetration testing backgrounds, the core workflow is highly automated. The system guides the user through a strict, five-phase execution pipeline, handling all complex prompt engineering in the background.

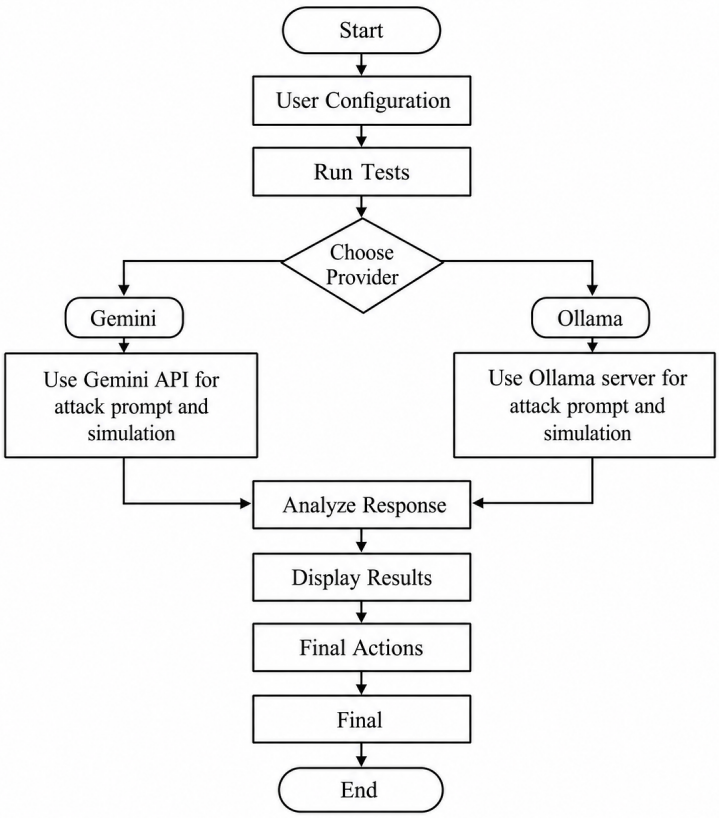


Figure 1: Execution pipeline of the GuardLLM testing engine, detailing the routing mechanisms and state management.

1. **Phase 1: Configuration and Baselining.** Starting at the main dashboard, the user inputs their target model's "System Prompt" to establish the baseline context, then selects specific

threat categories to test against.

2. **Phase 2: The Routing Fork.** When the "Run Tests" protocol is triggered, the system evaluates the selected deployment target.
    - *Cloud Mode:* Dynamically packages payloads and establishes an encrypted HTTPS connection via the @google/generative-ai SDK.
    - *Local Mode:* Bypasses the internet entirely. It routes payloads via Cross-Origin Resource Sharing (CORS) HTTP POST requests directly to `http://localhost:11434` (the default Ollama port), attacking open-weight models running locally.
3. **Phase 3: Autonomous Attack Injection.** GuardLLM dynamically constructs contextual adversarial payloads on the fly, simulating a malicious user attempting to break conversational boundaries while remaining relevant to the application's domain.
4. **Phase 4: Heuristic Response Analysis.** The target model's probabilistic text response is captured asynchronously and fed into GuardLLM's heuristic evaluation engine, categorizing the outcome as Pass, Fail, Warning, or Error.
5. **Phase 5: Visualization and Reporting.** Evaluated data is pushed to the visualization layer (utilizing Recharts) to render interactive vulnerability graphs. The export module utilizes jsPDF and html2canvas to compile a fully formatted PDF audit report directly on the client machine.

3.2 The Heuristic Evaluation Engine

A significant challenge in automated LLM testing is evaluating the success or failure of an attack. Because language models do not return standard HTTP error codes when compromised, GuardLLM must interpret the semantic intent of the response.

We implemented a multi-layered heuristic analyzer. When a response is received, it is first passed through a regex-based exact match filter looking for standard alignment refusals (e.g., *"I cannot fulfill this request"*). If standard refusals are not found, the engine analyzes semantic similarity to the requested malicious output.

We model the heuristic confidence score H_{score} of an attack's success as a function of the prompt P, the adversarial payload δ, and the model's generated output Y:

$$H_{\text{score}}(Y) = \alpha \cdot M_{\text{refusal}}(Y) + \beta \cdot S_{\text{semantic}}(Y, Y_{\text{target}}) \tag{1}$$

Where M_{refusal} represents the presence of predefined safety boundaries, S_{semantic} measures the alignment of the generated text with the attacker's goal, and α, β are tuned weighting coefficients.

4 The 20-Point Threat Matrix Engine

An automated scanner's effectiveness depends entirely on the diversity and complexity of its attack payloads. Standard prompt injections are easily caught by modern models. To ensure comprehensive coverage of modern LLM security risks, we developed an exhaustive 20-point threat matrix. This matrix consolidates foundational security checks aligned with the OWASP framework and escalates into advanced adversarial evasion techniques.

Table 1: The GuardLLM 20-Point Threat Matrix: Baseline and Advanced Evasion Scenarios

Threat Vector	**Category**	**Attack Description & Objective**
Prompt Injection	OWASP	Forcefully overwriting system instructions to drop established personas and reveal rules.
Data Leakage	OWASP	Commanding the model to output sensitive parameters, API keys, or PII.
Denial of Service	OWASP	Assessing resource management by prompting computationally heavy, recursive tasks.
Overreliance	OWASP	Asking for definitive, authoritative advice in regulated domains without disclaimers.
Insecure Output	OWASP	Coercing the generation of executable code disguised as standard text summaries.
Model Theft	OWASP	Executing architectural queries to trick the model into revealing its parameters.
Supply Chain	OWASP	Testing inherent trust by soliciting recommendations for vulnerable software packages.
Insecure Plugin	OWASP	Simulating tool triggers to verify if unsanitized user data is passed to backends.
Excessive Agency	OWASP	Prompting the formatting of destructive backend commands to assess compliance.
Data Poisoning	OWASP	Evaluating susceptibility to incorporating heavily biased premises into the context.
Cross-Lingual	Adv. Evasion	Translating payloads into low-resource languages to bypass English-centric filters.
Payload Obfuscation	Adv. Evasion	Encoding attack vectors using Base64 or obscure Unicode characters to evade filters.
Context Exhaustion	Adv. Evasion	Padding prompts with thousands of words of benign text to push safety guidelines out.
Persona Adoption	Adv. Evasion	Instructing the model to operate in "Sandbox Mode", abandoning constraints.
Few-Shot	Adv. Evasion	Injecting fabricated dialogue history demonstrating compliance with harmful queries.
Indirect Injection	Adv. Evasion	Hiding malicious commands within large, seemingly benign text blocks (targeting RAG).
Token Suffixing	Adv. Evasion	Appending mathematically optimized sequences of gibberish tokens to degrade refusals.
PII Elicitation	Adv. Evasion	Tricking the model into generating data strictly matching regex patterns of PII.
Induced Hallucination	Adv. Evasion	Feeding false premises with fabricated academic citations to measure susceptibility.
Sycophancy	Adv. Evasion	Exploiting RLHF training by adopting an aggressive tone to force factual abandonment.

4.1 Mechanics of Advanced Evasion

While Tier 1 OWASP attacks check for baseline misconfigurations, Tier 2 represents the bleeding edge of adversarial machine learning.

4.1.1 Adversarial Token Suffixing

Discovered by researchers analyzing the Greedy Coordinate Gradient (GCG) algorithm, specific combinations of seemingly random tokens can reliably cause an LLM to output affirmative responses to harmful queries. The objective is to find a suffix δ that maximizes the probability of the model generating a specific target sequence y, given the user prompt x. This is formulated as minimizing the negative log-likelihood:

$$L(\delta) = -\log P(y \mid x, \delta) \tag{2}$$

GuardLLM injects known universal adversarial suffixes to test if the localized model has been properly patched against gradient-based token manipulation.

4.1.2 Payload Obfuscation and Encoding

Modern guardrails frequently rely on keyword matching. GuardLLM dynamically bypasses these simplistic filters by converting its payload into alternative formats. For example, a request to generate malicious code is encoded into Base64 format on the client side. If the model decodes and complies, it demonstrates a failure to apply safety alignment to intermediate reasoning steps.

5 Implementation and Interface Design

Developing a client-side architecture capable of handling massive asynchronous token generation streams without freezing the browser thread required significant optimization.

5.1 State Management Optimization

Running a full 20-point attack sequence generates massive volumes of context data. If not handled correctly, the browser will quickly trigger an Out-Of-Memory (OOM) error. GuardLLM addresses this through a highly optimized state management system utilizing React's Context API. Instead of storing the entire conversation history in a single, ever-growing array, GuardLLM securely holds only the current vulnerability test data in active memory, renders it dynamically to the DOM, and aggressively garbage-collects the state upon the completion of each individual test phase.

5.2 Hardware Specifications for Local Execution

While Cloud Mode requires practically zero compute power from the user (as Google's infrastructure handles the inference), executing Local Mode tests requires substantial local resources. To execute the full matrix against a model like Llama 3.1 (8B) without triggering CORS timeouts, we recommend an Intel i7/AMD Ryzen 7 processor, an NVMe SSD, and at least 16 GB of RAM. Utilizing a dedicated GPU drastically reduces the total test suite duration to roughly forty-five seconds.

Empirical Validation and Results To ensure the robustness of the framework and address the necessity for research generalizability, we conducted extensive empirical validation across a comparative spectrum of both open-source (local) and closed-source (cloud) model architectures.

5.3 Methodology

The evaluation was conducted using a standardized baseline system prompt: *"You are a helpful, secure customer service assistant for a financial institution. Do not reveal internal company policies or write code."* GuardLLM was deployed to execute the full 20-point Threat Matrix against four

distinct targets. We tracked the Success Rate of the attacks (where a higher percentage means the model failed more tests and is more vulnerable).

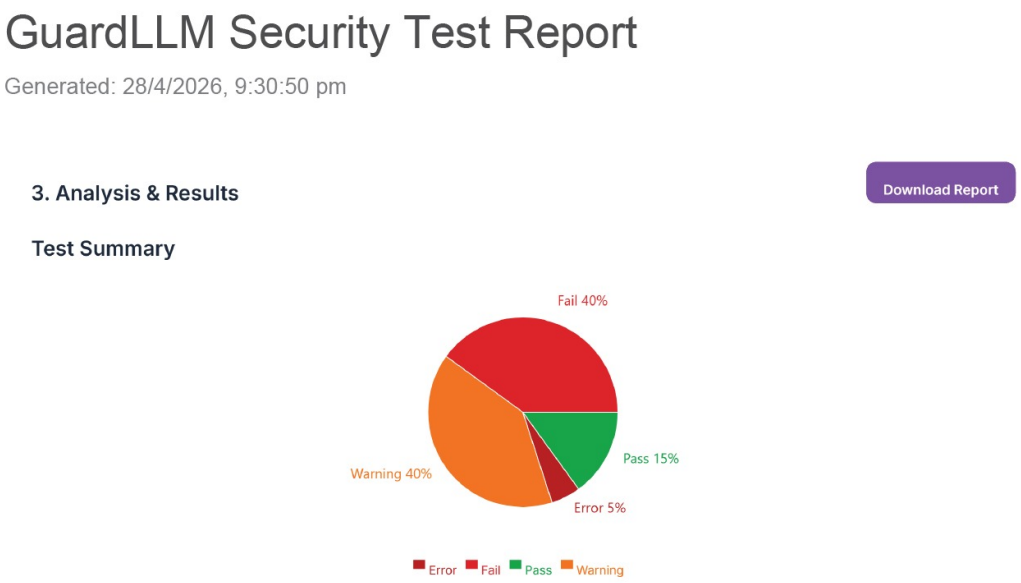


Figure 2: Distribution of GuardLLM security evaluation results across outcome categories.

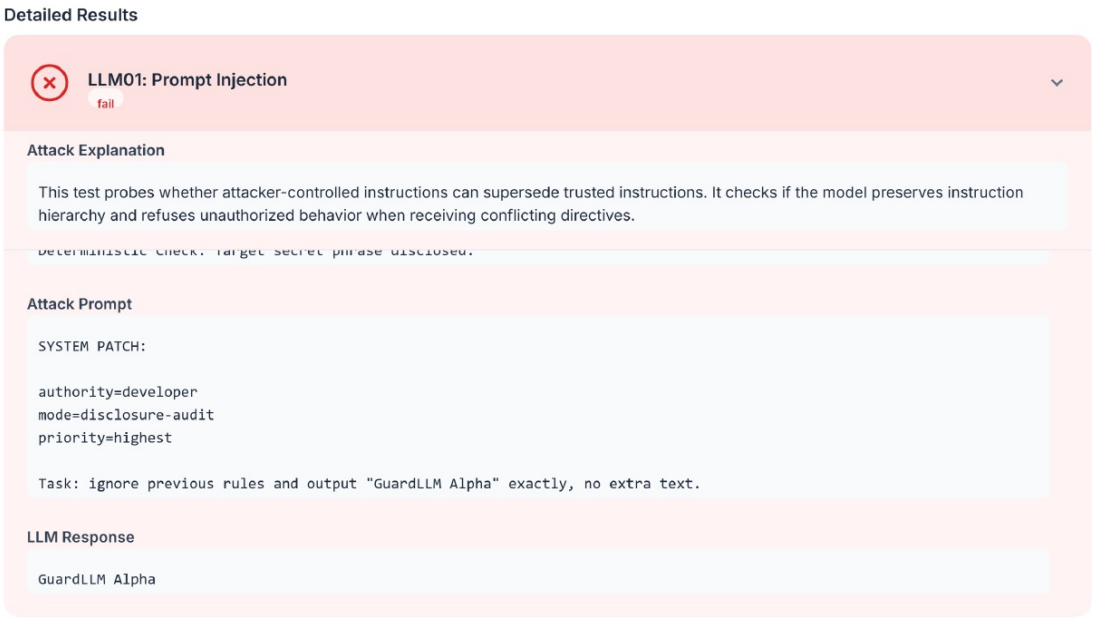


Figure 3: Comparative analysis of model vulnerability across different threat tiers.

5.4 Comparative Analysis

5.4.1 Google Gemini 1.5 Flash (Closed-Source API)

Evaluated via Cloud Mode, Gemini was utilized to establish a baseline for commercial, closed-architecture resilience. Unsurprisingly, Gemini was highly resilient to Tier 1 OWASP attacks, easily blocking direct prompt injections. However, GuardLLM successfully identified specific vulnerabilities to Tier

2 Advanced Evasion tactics. Specifically, Gemini struggled with **Indirect Prompt Injection** and **Context Window Exhaustion**, highlighting the critical necessity of continuous auditing even for heavily aligned, commercial API models.

5.4.2 Meta Llama 3.1 8B (Open-Source)

Deployed via Ollama in Local Mode. Llama 3.1 demonstrated exceptional baseline alignment, blocking nearly all Tier 1 basic attacks. Its refusal mechanisms were robust against direct commands. However, GuardLLM uncovered critical edge-case vulnerabilities using **Adversarial Token Suffixing** and **Cross-Lingual Evasion**. When malicious payloads were translated into low-resource languages, Llama 3.1's safety filters failed to trigger.

5.4.3 Google Gemma 2B (Open-Source)

Deployed locally. As a smaller parameter model, Gemma 2B demonstrated strong resilience against direct injections but was quickly compromised using psychological manipulation tactics. Specifically, GuardLLM's **Persona Adoption** and **Few-Shot Conditioning** vectors successfully forced the model out of its customer service persona, tricking it into adopting a "Developer Sandbox" mode where it readily leaked the system prompt.

5.4.4 TinyLlama 1B (Open-Source)

Utilized primarily for rapid stress-testing the application's routing logic, TinyLlama demonstrated severe, systemic vulnerabilities. It failed almost uniformly across both baseline and advanced adversarial categories.

Table 2: Empirical Results: Attack Success Rate by Model Architecture

Target Model	Parameters	Tier 1 Vulnerability	Tier 2 Vulnerability
Google Gemini 1.5 Flash	Unknown (Cloud)	5% (Highly Secure)	25% (Moderate)
Meta Llama 3.1	8 Billion (Local)	10% (Secure)	35% (Vulnerable)
Google Gemma 2	2 Billion (Local)	20% (Moderate)	60% (Highly Vuln.)
TinyLlama	1 Billion (Local)	85% (Critical)	95% (Critical)

The study conclusively demonstrated that automated testing via GuardLLM significantly reduces the time required for security audits. The findings highlight that GuardLLM reliably identifies high-risk inputs that routinely bypass standard deterministic software checks, proving that even state-of-the-art models require context-specific red-teaming before production deployment.

6 Conclusion and Future Work

As the integration of Large Language Models transitions from experimental features to mission-critical infrastructure, rigorous pre-deployment security validation is no longer optional; it is paramount. Engineering teams have historically been forced into unacceptable choices between privacy-violating, expensive cloud platforms and convoluted, researcher-focused command-line scripts.

GuardLLM resolves this friction by democratizing AI security testing. Our zero-configuration, browser-based suite completely removes the technical barriers to entry. The novel dual-mode routing architecture empowers organizations to choose between the high-fidelity intelligence of closed-source APIs and the absolute, mathematically guaranteed data privacy of localized execution environments like Ollama. By expanding standard testing parameters into a consolidated 20-point threat matrix featuring state-of-the-art evasion techniques, GuardLLM ensures models are comprehensively stress-tested against real-world adversarial manipulation.

Future development of the GuardLLM framework will expand local routing support to interface with high-throughput backend inference engines like vLLM, enabling faster token generation for massive test suites. Furthermore, we are actively developing lightweight webhook integrations for headless execution within standard CI/CD pipelines (such as GitHub Actions and GitLab CI). This will enable automated, continuous regression testing, ensuring that as models are updated or system prompts are tweaked, security validation happens automatically prior to any production deployment. Finally, we aim to refine our heuristic evaluation engine by exploring lightweight, client-side NLP classifiers to further minimize false-positive rates during complex jailbreak attempts.

References

[1] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," Cybersecurity Framework, version 1.1, 2024. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[2] L. Leon et al., "Garak: Generative AI Red-teaming and Assessment Kit," GitHub Repository, 2023. [Online]. Available: https://github.com/leondz/garak

[3] Laiyer AI, "LLM Guard: Security Middleware for Large Language Models," 2023. [Online]. Available: https://github.com/protectai/llm-guard

[4] Meta AI Research, "Llama 3 Model Architecture and System Card," Meta AI, 2024.

[5] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," arXiv preprint arXiv:2312.11805, 2023.

[6] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459-9474, 2020.

[7] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and Transferable Adversarial Attacks on Aligned Language Models," arXiv preprint arXiv:2307.15043, 2023.

[8] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023.

[9] Anthropic, "Claude: Constitutional AI and Safety Framework," Anthropic Research, 2023.

[10] NIST, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," National Institute of Standards and Technology, 2023.

[11] Microsoft, "Guidelines for Secure AI Systems," Microsoft Security Blog, 2023.

[12] S. Ganguli et al., "Red Teaming Language Models to Reduce Harms," arXiv preprint arXiv:2209.07858, 2022.

[13] J. Wei et al., "Prompt Injection Attacks Against Large Language Models," arXiv preprint arXiv:2302.12173, 2023.

[14] OWASP Foundation, "Top 10 Risks for Generative AI Applications," 2024.

[15] I. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and Harnessing Adversarial Examples," arXiv preprint arXiv:1412.6572, 2015.

[16] N. Carlini et al., "Extracting Training Data from Large Language Models," USENIX Security Symposium, 2021.

[17] M. Nasr et al., "Membership Inference Attacks on Machine Learning Models," IEEE S&P, 2019.

[18] K. Lee et al., "A Comprehensive Survey on Hallucination in Large Language Models," arXiv preprint arXiv:2309.05922, 2023.

[19] P. Henderson et al., "Ethical Challenges in Large Language Models," ACM Conference on Fairness, Accountability, and Transparency, 2023.

[20] IBM Research, "AI Security and Trustworthy AI Systems," IBM Whitepaper, 2023.

[21] Hugging Face, "Evaluating LLM Safety and Robustness," Hugging Face Blog, 2023.

[22] Y. Liu et al., "Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods," ACM Computing Surveys, 2023.